

Tarea Semana 1: Sistema de Calificaciones

Condicionales y Control de Flujo

Curso de Programación en Python

Tiempo de entrega: 1 semana

1. Introducción

¡Bienvenido a la segunda semana del curso! Ya conoces los fundamentos de Python: variables, funciones básicas y tipos de datos. Ahora es momento de aprender a tomar decisiones en tu código.

Los condicionales son el corazón de la lógica de programación. Permiten que tu programa tome diferentes caminos según las condiciones que establezcas. Es como cuando decides qué ropa ponerte según el clima, o qué ruta tomar según el tráfico.

En esta tarea, crearás un sistema que evalúa calificaciones y toma decisiones basadas en diferentes criterios. Aprenderás a usar `if`, `elif`, `else`, operadores de comparación y lógicos para crear un programa inteligente que responda a diferentes situaciones.

2. Objetivos de Aprendizaje

Al completar esta tarea, serás capaz de:

- Usar operadores de comparación: `>`, `<`, `>=`, `<=`, `==`, `!=`
- Implementar estructuras condicionales: `if`, `elif`, `else`
- Combinar condiciones con operadores lógicos: `and`, `or`, `not`
- Trabajar con expresiones booleanas y valores `True/False`
- Anidar condicionales cuando sea necesario
- Validar entradas del usuario con condiciones
- Usar el operador `match` (Python 3.10+) para selección múltiple
- Crear programas que tomen decisiones complejas

3. Enunciado del Proyecto

3.1. Descripción General

Vas a crear un programa llamado “**Sistema de Calificaciones Académicas del TEC**”. Este sistema ayudará a estudiantes a calcular su nota final en un curso y determinará si aprueban, reprueban, necesitan convocatoria, o si obtienen una mención de honor.

El sistema debe solicitar diferentes calificaciones, calcular promedios, aplicar reglas académicas específicas, y dar retroalimentación personalizada según el rendimiento del estudiante.

3.2. Contexto Académico

En el sistema académico del TEC (y de la mayoría de universidades costarricenses):

- La nota mínima para aprobar es **70**
- La nota máxima es **100**
- Entre 65 y 69 se puede ir a **convocatoria**
- Menos de 65 es **reprobado**
- 90 o más es **excelente** (mención de honor)
- Se pueden tener diferentes componentes: exámenes, proyectos, tareas, etc.

4. Requisitos Funcionales

Tu programa debe realizar las siguientes acciones:

4.1. Parte 1: Recopilación de Información

1. Datos del estudiante:

- Solicitar nombre del estudiante
- Solicitar nombre del curso

2. Componentes de calificación:

- Nota de exámenes (40 % de la nota final)
- Nota de proyecto (30 % de la nota final)
- Nota de tareas (20 % de la nota final)
- Nota de participación (10 % de la nota final)

3. Preguntar si hubo ausencias:

- Cantidad de ausencias en el semestre
- Si tiene más de 3 ausencias, hay penalización

4.2. Parte 2: Validaciones

Tu programa debe validar que:

1. Todas las notas estén en el rango válido:

- Cada nota debe estar entre 0 y 100
- Si alguna nota es inválida, mostrar mensaje de error y terminar

2. El nombre no esté vacío:

- Verificar que el estudiante ingresó su nombre
- Verificar que ingresó el nombre del curso

3. Las ausencias sean un número válido:

- No pueden ser negativas
- Deben ser un número entero

4.3. Parte 3: Cálculos y Determinaciones

1. Calcular la nota final:

- Aplicar los porcentajes correctos a cada componente
- Sumar para obtener la nota final

2. Aplicar penalización por ausencias:

- Si tiene más de 3 ausencias: restar 5 puntos a la nota final
- La nota nunca puede ser menor que 0

3. Determinar el estado académico:

- $\text{Nota} \geq 90$: “Aprobado con Excelencia”
- $\text{Nota} \geq 70$: “Aprobado”
- Nota entre 65 y 69: “Convocatoria”
- $\text{Nota} < 65$: “Reprobado”

4. Determinar el componente más fuerte y más débil:

- Identificar en cuál componente tuvo la mejor nota
- Identificar en cuál componente tuvo la peor nota

5. Dar recomendaciones personalizadas:

- Basadas en el rendimiento
- Basadas en las ausencias
- Basadas en los componentes débiles

4.4. Parte 4: Presentación de Resultados

El programa debe mostrar:

- Resumen de notas por componente
- Nota final calculada (con 2 decimales)
- Estado académico (aprobado, convocatoria, reprobado, excelencia)
- Componente más fuerte y más débil
- Recomendaciones personalizadas
- Si hubo penalización por ausencias

4.5. Ejemplo de Ejecución 1: Estudiante Aprobado

Listing 1: Ejemplo - Estudiante aprobado con buenas notas

```
1  === SISTEMA DE CALIFICACIONES ACADEMICAS ===
2
3  --- INFORMACION DEL ESTUDIANTE ---
4  Nombre del estudiante: Maria Fernandez
5  Nombre del curso: Programacion en Python
6
7  --- INGRESE LAS CALIFICACIONES ---
8  Nota de exámenes (40%): 85
9  Nota de proyecto (30%): 90
10 Nota de tareas (20%): 88
11 Nota de participacion (10%): 95
12
13 Cantidad de ausencias en el semestre: 2
14
15 --- PROCESANDO CALIFICACIONES ---
16
17 =====
18      REPORTE DE CALIFICACIONES
19 =====
20
21 Estudiante: Maria Fernandez
22 Curso: Programacion en Python
23
24 --- Desglose de Notas ---
25 Exámenes (40%):      85.00
26 Proyecto (30%):      90.00
27 Tareas (20%):        88.00
28 Participacion (10%):  95.00
29
30 --- Resultado Final ---
31 Nota Final: 87.70
32 Estado: APROBADO
33
34 --- Analisis de Rendimiento ---
35 Componente mas fuerte: Participacion (95.00)
36 Componente mas debil: Exámenes (85.00)
37
38 --- Observaciones ---
39 - Felicidades! Has aprobado el curso.
40 - Tu participacion fue excelente, sigue asi!
41 - Podrias mejorar en Exámenes para la proxima.
42 - Tienes un buen record de asistencia.
43
44 =====
```

4.6. Ejemplo de Ejecución 2: Estudiante en Convocatoria

Listing 2: Ejemplo - Estudiante que va a convocatoria

```
1  === SISTEMA DE CALIFICACIONES ACADEMICAS ===
2
3  --- INFORMACION DEL ESTUDIANTE ---
4  Nombre del estudiante: Carlos Rodriguez
5  Nombre del curso: Calculo Diferencial
6
7  --- INGRESE LAS CALIFICACIONES ---
8  Nota de exámenes (40%): 65
9  Nota de proyecto (30%): 70
10 Nota de tareas (20%): 68
11 Nota de participacion (10%): 75
12
13 Cantidad de ausencias en el semestre: 5
14
15 --- PROCESANDO CALIFICACIONES ---
16 ATENCION: Penalizacion de 5 puntos por exceso de ausencias!
17
18 =====
19      REPORTE DE CALIFICACIONES
20 =====
21
22 Estudiante: Carlos Rodriguez
23 Curso: Calculo Diferencial
24
25 --- Desglose de Notas ---
26 Exámenes (40%):      65.00
27 Proyecto (30%):      70.00
28 Tareas (20%):        68.00
29 Participacion (10%):  75.00
30
31 --- Calculo ---
32 Nota antes de penalizacion: 68.10
33 Penalizacion por ausencias: -5.00
34 Nota Final: 63.10
35
36 Estado: REPROBADO
37
38 --- Analisis de Rendimiento ---
39 Componente mas fuerte: Participacion (75.00)
40 Componente mas debil: Exámenes (65.00)
41
42 --- Observaciones ---
43 - Lamentablemente has reprobado el curso.
44 - Las ausencias afectaron significativamente tu nota.
45 - Debes mejorar tu asistencia a clases.
46 - Tu componente mas debil es Exámenes, dedica mas tiempo.
47 - Recomendamos tomar un curso de nivelacion.
48
49 =====
```

5. Requisitos Técnicos Detallados

5.1. Uso de Condicionales

Tu programa DEBE incluir:

1. Validación con if:

```
1 # Ejemplo: Validar rango de notas
2 if nota_examen < 0 or nota_examen > 100:
3     print("Error: La nota debe estar entre 0 y 100")
```

2. Múltiples condiciones con elif:

```
1 # Ejemplo: Determinar estado academico
2 if nota_final >= 90:
3     estado = "Aprobado con Excelencia"
4 elif nota_final >= 70:
5     estado = "Aprobado"
6 elif nota_final >= 65:
7     estado = "Convocatoria"
8 else:
9     estado = "Reprobado"
```

3. Operadores lógicos (and, or, not):

```
1 # Ejemplo: Verificar multiples condiciones
2 if ausencias > 3 and nota_final > 0:
3     nota_final -= 5 # Aplicar penalizacion
4
5 if nombre == "" or curso == "":
6     print("Error: Debe ingresar el nombre")
```

4. Condicionales anidados:

```
1 # Ejemplo: Dar recomendaciones especificas
2 if estado == "Aprobado":
3     if nota_final >= 90:
4         print("Felicidades por tu excelencia!")
5     else:
6         print("Buen trabajo!")
7 else:
8     if ausencias > 3:
9         print("Las ausencias afectaron tu nota")
```

5. Operador match (OPCIONAL - Python 3.10+):

```
1 # Ejemplo: Match para categorizar estado
2 match estado:
3     case "Aprobado con Excelencia":
4         print("Rendimiento sobresaliente!")
5     case "Aprobado":
6         print("Cumpliste con los objetivos!")
7     case "Convocatoria":
```

```
8         print("Tienes una oportunidad mas!")
9     case "Reprobado":
10         print("Debes mejorar para la proxima")
```

5.2. Estructura del Código

■ Comentarios descriptivos:

- Encabezado con nombre, fecha y descripción
- Comentarios explicando cada sección lógica
- Comentarios en validaciones complejas

■ Variables con nombres claros:

- `nota_exámenes`, `nota_proyecto`, etc.
- `nota_final`, `estado_academico`
- `componente_fuerte`, `componente_debil`

■ Conversión de tipos apropiada:

- `float()` para las notas
- `int()` para las ausencias
- `.strip()` para limpiar nombres

■ Formato profesional:

- Usar f-strings para todos los mensajes
- Mostrar notas con 2 decimales: `:.2f`
- Líneas separadoras para organizar la salida
- Mensajes claros y bien redactados

6. Consejos y Recomendaciones

- **Planifica la lógica primero:** Dibuja un diagrama de flujo o escribe pseudocódigo antes de programar.
- **Valida temprano:** Verifica las entradas del usuario antes de hacer cálculos.
- **Prueba todos los casos:** Asegúrate de probar:
 - Estudiante aprobado ($\text{nota} \geq 70$)
 - Estudiante con excelencia ($\text{nota} \geq 90$)
 - Estudiante en convocatoria (65-69)
 - Estudiante reprobado ($\text{nota} < 65$)
 - Con y sin penalización por ausencias
 - Notas inválidas (fuera de rango)
- **Cuida el orden de los condicionales:** En estructuras `if-elif-else`, el orden importa.

- **Usa paréntesis para claridad:** Cuando combines condiciones, usa paréntesis para evitar errores.
- **No repitas código:** Si calculas algo más de una vez, guárdalo en una variable.
- **Mensajes amigables:** Los mensajes de error deben ser claros y ayudar al usuario.

7. Entregables

1. Archivo Python llamado `sistema_calificaciones.py`
2. El código debe ejecutarse sin errores
3. Debe incluir todos los comentarios requeridos
4. Debe manejar todos los casos descritos

8. Defensa del Código (60 % de la nota final)

8.1. ¿Qué es la defensa del código?

La defensa del código es una reunión virtual donde presentarás y explicarás tu trabajo. Esta defensa es **obligatoria** y representa el **60 % de la nota final** de esta tarea.

8.2. ¿Cómo funciona?

Durante la defensa deberás:

1. **Compartir tu pantalla** mostrando tu código en el editor.
2. **Ejecutar tu programa** demostrando diferentes casos:
 - Un estudiante que apruebe
 - Un estudiante que repruebe
 - Un estudiante en convocatoria
 - Un estudiante con penalización por ausencias
 - Validación de datos inválidos
3. **Explicar tu código:**
 - Cómo funcionan tus validaciones
 - Por qué usaste `if-elif-else` en ciertos lugares
 - Cómo determinaste el componente más fuerte/débil
 - Cómo calculaste la nota final con porcentajes
 - Qué operadores lógicos usaste y por qué
4. **Responder preguntas como:**
 - “¿Qué pasaría si cambias este `>=` por `>`?”
 - “¿Por qué usaste `and` aquí y no `or`?”

- “¿Cómo modificarías el código para agregar otro componente?”
- “¿Qué pasa si las ausencias son 3 exactamente?”
- “¿Por qué validaste las notas antes de calcular?”

5. Demostrar que entiendes la lógica:

- Explica cómo fluye el programa según diferentes entradas
- Describe qué condiciones llevan a qué resultados
- Justifica tus decisiones de diseño

8.3. Preguntas Típicas de la Defensa

Prepárate para responder:

- ¿Por qué validaste primero antes de calcular?
- ¿Qué diferencia hay entre usar `==` y `=`?
- ¿Cuál es la diferencia entre `and` y `or`?
- Si cambio esta condición, ¿qué pasaría?
- ¿Cómo decides entre usar `if-elif` o varios `if` separados?
- ¿Por qué es importante el orden en los `elif`?
- ¿Qué hace el operador `not`?
- ¿Cómo probaste que tu programa funciona correctamente?

8.4. Consejos para una Buena Defensa

- **Practica explicar tu código** a un amigo o en voz alta
- **Entiende cada condicional** que escribiste
- **Ten ejemplos preparados** con diferentes valores
- **Conoce los operadores** de comparación y lógicos
- **Sé honesto:** Si usaste ayuda, menciónalo, pero asegúrate de entender todo

8.5. Distribución de la Nota

- **Código funcional (40 %):** El programa funciona según los requisitos
- **Defensa del código (60 %):** Tu capacidad de explicar y defender tu trabajo

Importante: Si no asistes a la defensa o no puedes explicar tu código, la nota máxima será 40/100.

9. Desafíos Opcionales (Puntos Extra)

1. Múltiples intentos (+5 pts):

- Si el usuario ingresa una nota inválida, permitir reintentar
- No terminar el programa, dar 3 oportunidades

2. Tabla de honor (+4 pts):

- Si la nota es ≥ 95 , agregar a una “Tabla de Honor”
- Mostrar un mensaje especial de felicitación

3. Estadísticas adicionales (+4 pts):

- Calcular el promedio de todos los componentes
- Mostrar cuántos componentes están arriba de 80

4. Sistema de créditos (+5 pts):

- Preguntar cuántos créditos tiene el curso (1-4)
- Calcular el impacto en el promedio general del estudiante
- Dar recomendaciones según la cantidad de créditos

5. Uso de match (+3 pts):

- Implementar al menos un `match` de forma apropiada
- Por ejemplo, para categorizar mensajes según el estado

Nota: Los puntos extra solo se otorgan si la implementación es correcta Y puedes explicarla en la defensa.

10. Rúbrica de Evaluación

10.1. Distribución General

- **Código Funcional:** 40 % (40 puntos)
- **Defensa del Código:** 60 % (60 puntos)
- **Total:** 100 puntos
- **Puntos Extra:** Hasta +21 puntos adicionales

10.2. Parte 1: Código Funcional (40 puntos)

10.2.1. 1.1 Funcionalidad Básica (15 puntos)

- **Ejecución sin errores (5 pts):**
 - 5 pts: Programa ejecuta perfectamente en todos los casos
 - 3 pts: Ejecuta con errores menores en casos específicos
 - 1 pt: Ejecuta pero con errores frecuentes
 - 0 pts: No ejecuta o crashea
- **Recopilación de datos (5 pts):**
 - 5 pts: Solicita todos los datos requeridos (nombre, curso, 4 notas, ausencias)
 - 3 pts: Solicita la mayoría de datos, falta alguno
 - 1 pt: Solicita solo algunos datos
 - 0 pts: No solicita datos apropiadamente
- **Cálculo de nota final (5 pts):**
 - 5 pts: Calcula correctamente con porcentajes (40 %, 30 %, 20 %, 10 %)
 - 3 pts: Calcula pero con algún porcentaje incorrecto
 - 1 pt: Intenta calcular pero lógica incorrecta
 - 0 pts: No calcula o completamente incorrecto

10.2.2. 1.2 Validaciones (10 puntos)

- **Validación de rango de notas (4 pts):**
 - 4 pts: Valida que todas las notas estén entre 0 y 100
 - 3 pts: Valida la mayoría de notas
 - 1 pt: Validación incompleta o incorrecta
 - 0 pts: No valida rangos
- **Validación de nombres (3 pts):**
 - 3 pts: Verifica que nombre y curso no estén vacíos
 - 2 pts: Valida solo uno de los dos
 - 1 pt: Validación parcial

- 0 pts: No valida nombres

■ **Validación de ausencias (3 pts):**

- 3 pts: Verifica que sean no negativas y entero válido
- 2 pts: Validación parcial
- 1 pt: Intenta validar pero con errores
- 0 pts: No valida ausencias

10.2.3. 1.3 Uso de Condicionales (15 puntos)

■ **Determinación de estado académico (5 pts):**

- 5 pts: Usa `if-elif-else` correctamente para 4 estados
- 4 pts: Categoriza correctamente 3 de 4 estados
- 2 pts: Lógica parcialmente correcta
- 0 pts: No determina estado o lógica incorrecta

■ **Penalización por ausencias (3 pts):**

- 3 pts: Aplica -5 puntos si ausencias ≥ 3 , previene nota ≤ 0
- 2 pts: Aplica penalización pero sin verificación de mínimo
- 1 pt: Lógica parcial
- 0 pts: No implementa penalización

■ **Identificación de componentes fuerte/débil (4 pts):**

- 4 pts: Identifica correctamente ambos componentes
- 3 pts: Identifica uno correctamente
- 1 pt: Intenta pero lógica incorrecta
- 0 pts: No identifica componentes

■ **Recomendaciones personalizadas (3 pts):**

- 3 pts: Da recomendaciones basadas en múltiples factores
- 2 pts: Recomendaciones básicas pero relevantes
- 1 pt: Recomendaciones genéricas o mínimas
- 0 pts: No da recomendaciones

10.3. Parte 2: Defensa del Código (60 puntos)

10.3.1. 2.1 Comprensión de Condicionales (25 puntos)

■ **Explicación de operadores de comparación (7 pts):**

- 7 pts: Explica perfectamente `>`, `<`, `>=`, `<=`, `==`, `!=`
- 5 pts: Explica correctamente la mayoría
- 3 pts: Explicación básica o con errores
- 0 pts: No entiende los operadores

■ Explicación de if-elif-else (8 pts):

- 8 pts: Explica claramente cómo fluye la ejecución según condiciones
- 6 pts: Explica correctamente con pequeñas imprecisiones
- 3 pts: Explicación superficial o confusa
- 0 pts: No entiende la estructura

■ Explicación de operadores lógicos (10 pts):

- 10 pts: Explica perfectamente **and**, **or**, **not** con ejemplos de su código
- 7 pts: Explica correctamente con alguna duda
- 4 pts: Explicación parcial o con errores
- 0 pts: No entiende operadores lógicos

10.3.2. 2.2 Demostración y Razonamiento (20 puntos)**■ Ejecución de diferentes casos (8 pts):**

- 8 pts: Demuestra exitosamente 4+ casos diferentes
- 6 pts: Demuestra 3 casos correctamente
- 3 pts: Demuestra 2 casos
- 0 pts: No puede ejecutar múltiples casos

■ Respuesta a preguntas técnicas (12 pts):

- 12 pts: Responde correcta y claramente todas las preguntas sobre lógica
- 9 pts: Responde bien la mayoría, algunas dudas
- 5 pts: Respuestas parciales o confusas
- 2 pts: Dificultad con preguntas básicas
- 0 pts: No puede responder sobre su lógica

10.3.3. 2.3 Calidad del Código (15 puntos)**■ Comentarios explicativos (5 pts):**

- 5 pts: Comentarios claros en encabezado y secciones lógicas clave
- 3 pts: Comentarios presentes pero básicos
- 1 pt: Comentarios mínimos
- 0 pts: Sin comentarios

■ Nombres de variables (5 pts):

- 5 pts: Todos los nombres son descriptivos y claros
- 3 pts: Mayoría de nombres buenos, algunos confusos
- 1 pt: Nombres poco claros
- 0 pts: Nombres inapropiados (x, y, var1, etc.)

■ Presentación y formato (5 pts):

- 5 pts: Salida profesional, bien organizada, con f-strings y formato correcto
- 3 pts: Salida aceptable, formato parcial
- 1 pt: Salida básica, poco formato
- 0 pts: Salida confusa o sin formato

10.4. Puntos Extra (Hasta +21 puntos)

- **Múltiples intentos (+5 pts):** Permite reintentar entradas inválidas
- **Tabla de honor (+4 pts):** Mensaje especial para notas ≥ 95
- **Estadísticas adicionales (+4 pts):** Calcula promedios y cuenta componentes ≥ 80
- **Sistema de créditos (+5 pts):** Calcula impacto según créditos del curso
- **Uso de match (+3 pts):** Implementa correctamente operador match

Nota: Los puntos extra solo se otorgan si la implementación es correcta Y el estudiante puede explicarla completamente durante la defensa.

10.5. Penalizaciones

- **-100 % de la defensa:** No asiste a la defensa sin justificación (nota máxima = 40/100)
- **-50 % de la defensa:** Evidencia clara de no entender el código (posible plagio)
- **-5 pts:** Entrega tardía de 1 día
- **-10 pts:** Entrega tardía de 2-3 días
- **-20 pts:** Entrega tardía de 4+ días
- **-3 pts:** Nombre de archivo incorrecto
- **-2 pts:** Falta encabezado con información del estudiante
- **-5 pts:** No valida ninguna entrada del usuario

10.6. Nota Mínima Aprobatoria

Para aprobar esta tarea necesitas obtener al menos **60 puntos de 100**.

10.7. Escala de Calificación

- **90-100+ pts:** Excelente - Dominio completo de condicionales
- **80-89 pts:** Muy Bueno - Comprensión sólida de control de flujo
- **70-79 pts:** Bueno - Comprensión aceptable, necesita práctica
- **60-69 pts:** Suficiente - Comprensión básica, necesita refuerzo
- **0-59 pts:** Insuficiente - No aprobado

11. Recursos de Apoyo

- **Documentación oficial:** <https://docs.python.org/es/3/tutorial/controlflow.html>
- **Operadores de comparación:** Revisa las notas de clase sobre comparaciones
- **Operadores lógicos:** Practica combinar condiciones con `and`, `or`, `not`
- **Tablas de verdad:** Entiende cómo funcionan los operadores lógicos
- **Debugging:** Usa `print()` para verificar el flujo de tu programa

12. Preguntas Frecuentes

P: ¿Cuál es la diferencia entre `=` y `==`?

R: `=` es asignación (guarda un valor), `==` es comparación (verifica si son iguales).

P: ¿Cuándo uso `and` vs `or`?

R: Usa `and` cuando AMBAS condiciones deben ser verdaderas. Usa `or` cuando AL MENOS UNA debe ser verdadera.

P: ¿Por qué el orden importa en `if-elif`?

R: Python evalúa de arriba hacia abajo y se detiene en la primera condición verdadera. Si pones condiciones en mal orden, podrías nunca llegar a algunas.

P: ¿Es obligatorio el `match`?

R: No, es opcional y da puntos extra. Puedes lograr todo con `if-elif-else`.

P: ¿Qué hago si no sé cómo encontrar el componente más fuerte?

R: Compara las 4 notas usando `if` para determinar cuál es la mayor.

P: ¿Puedo usar funciones?

R: Aún no hemos visto funciones formalmente. Para esta tarea, escribe todo en el flujo principal del programa.

¡Éxito en tu tarea de condicionales!

Los condicionales son la base de la lógica de programación.

Domínalos y podrás crear programas inteligentes y útiles.